

Optimism is an occupational hazard of programming: testing is the treatment.

- K. Beck

CSE 312

Operating Systems

Spring 2026

Scheduling

© 2013-2026 Yakup Genç

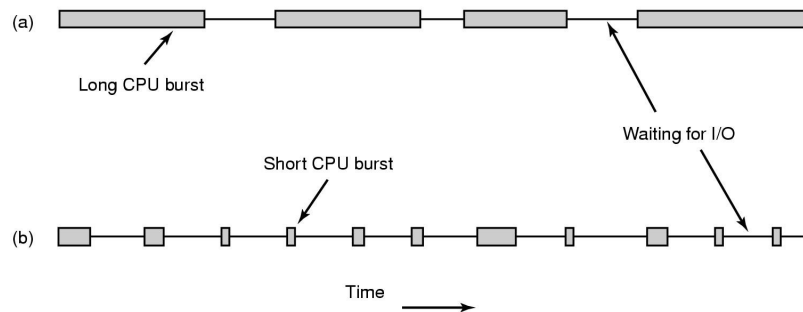
1

Today

- Scheduling

2

Scheduling – Process Behavior



Bursts of CPU usage alternate with periods of waiting for I/O

(a) A CPU-bound process

(b) An I/O-bound process

Spring 2026

Operating Systems

3

3

Scheduling

- Suppose several processes are runnable?
- Which one is run next?
- Many different ways to make this decision

Spring 2026

Operating Systems

4

4

Environments

- Old batch systems didn't have a scheduler; they just read whatever was next on the input tape
- Actually, they did have a scheduler: the person who loaded the card decks onto the tape
- Hybrid batch/time-sharing systems tend to give priority to short timesharing requests
- Still a policy today: must give priority to interactive requests

Spring 2026

Operating Systems

5

5

Process Behavior

- Processes alternate CPU use with I/O requests
- I/O requests frequently block, either waiting for input or when too much has been written and no buffer space is available
- CPU-bound processes think more than they read or write
- I/O-bound processes do lots of I/O; it's (usually) not that the I/O operations are so time-consuming
- Absolute speed of CPU and I/O devices is irrelevant; what matters is the ratio
- CPUs have been getting much faster relative to disks

Spring 2026

Operating Systems

6

6

When to Make Scheduling Decisions

- After a fork — run the parent or child?
- On process exit
- When a process blocks
- When I/O completes
- Sometimes, after timer interrupts

Spring 2026

Operating Systems

7

7

Preemptive vs. Nonpreemptive Schedulers

- Nonpreemptive scheduler: lets a process run as long as it wants
- Only switches when it blocks
- Preemptive: switches after a time quantum

Spring 2026

Operating Systems

8

8

Categories of Scheduling Algorithms

- Batch — responsiveness isn't important; preemption moderately important
- Interactive — must satisfy a human; preemption important
- Real-time — often nonpreemptive

Spring 2026

Operating Systems

9

9

Goals

- Fairness — give each process its share of the CPU
- Policy and enforcement — give preference to work that is administratively favored; prevent subversion of OS scheduling policy
- Balance — keep all parts of the system busy

Spring 2026

Operating Systems

10

10

Goals: Batch Systems

- Throughput — maximize jobs/hour
- Turnaround time — return jobs quickly. Often want to finish short jobs very quickly
- CPU utilization

Spring 2026

Operating Systems

11

11

Interactive Systems

- Response time — respond quickly to user requests
- Meet user expectations — psychological
 - Users have a sense of “cheap” and “expensive” requests
 - Users are happier if “cheap” requests finish quickly
 - “Cheap” and “expensive” don’t always correspond to reality!

Spring 2026

Operating Systems

12

12

Real-Time Systems

- Meet deadlines — avoid losing data (or worse!)
- Predictability — users must know when their requests will finish
- Requires careful engineering to match priorities to actual completion times and available resources

Spring 2026

Operating Systems

13

13

Batch Schedulers

- First-come, first-served
- Shortest first
- Shortest remaining time first
- Three-level scheduler

Spring 2026

Operating Systems

14

14

First-Come, First-Served

- Run the first process on the run queue
- Never preempt based on timer
- Seems simple; just like waiting in line
- Not very fair

Spring 2026

Operating Systems

15

15

First-Come, First-Served

- Imagine a CPU-bound process A: thinks for 1 second, then reads 1 disk block
- There's also an I/O-bound process B that needs to read 1000 blocks
- A runs for 1 second, then issues an I/O request
- B runs for almost no time, then issues an I/O request
- A then runs for another second
- It takes 1000 seconds for B to finish

Spring 2026

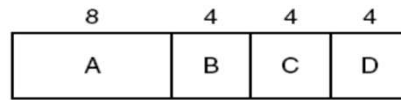
Operating Systems

16

16

Shortest First

- Suppose you know the time requirements of each job
- A needs 8 seconds, B needs 4, C needs 4, D needs 4
- Run B, C, D, A
- Nonpreemptive
- Provably fair:
 - Suppose four jobs have runtimes of a , b , c , and d
 - First finishes at time a , second at $a + b$, etc
 - Mean turnaround is $(4a + 3b + 2c + d)/4$
 - d contributes less to the mean



Spring 2026

Operating Systems

17

17

Optimality Requires Simultaneous Availability

- Jobs don't all arrive at the same time
- Can't make optimal scheduling decision without complete knowledge
- Example: jobs with times of 2,4,1,1,1 that arrive at times 0,0,3,3,3
- Shortest-first runs A, B, C, D, E; average wait is 4.6 secs
- If we run B, C, D, E, A, average wait is 4.4 secs
- While B is running, more jobs arrive, allowing a better decision for the total load

Spring 2026

Operating Systems

18

18

Shortest Remaining Time Next

- Preemptive variant of FCFS
- Still need to know run-times in advance
- Helps short jobs get good service
- May have a problem with indefinite overtaking

Spring 2026

Operating Systems

19

19

Three-Level Scheduler

- First stage: job queue
- Select different types of jobs (i.e., I/O- or CPU-bound) to balance workload
- Note: relies on humans classify jobs in advance
- Second stage: availability of main memory
 - Closely linked to virtual memory system; let's defer that
- CPU scheduler

Spring 2026

Operating Systems

20

20

User Requirements

- Users must be able to specify job characteristics: estimated CPU time, I/O versus CPU balance, perhaps memory
- Scheduler categories must reflect technical and managerial issues
- Lying about characteristics may give better turnaround times, but at the expense of total system throughput
- Should the response be technical or administrative?

Spring 2026

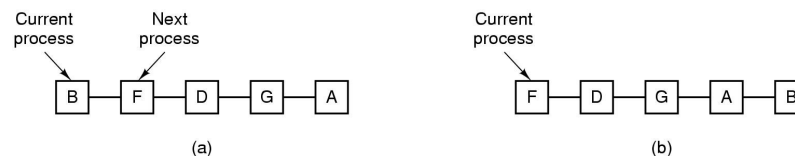
Operating Systems

21

21

Round-Robin

- Take turns
- Give each process a time quantum
- When the time is up, move the current process to the end of the queue



Spring 2026

Operating Systems

22

22

Quantum Length

- The shorter the quantum, the more responsive the system
- However, process switching is expensive (saving and reloading registers, switching virtual memory maps, flushing the cache, bookkeeping, etc.)
- Suppose the compute quantum is 4 msec and process switches take 1 msec.
- That's 20% overhead — too much
- Suppose we have 100 msec quanta
- If the run queue ever gets long, even cheap requests will take too long
- Need a reasonable compromise: 20-50 msec?

Spring 2026

Operating Systems

23

23

Priority Scheduling

- Not all processes are equally important
- Assign them priority levels
- Simplest version: always run the highest-priority process
- Not a good idea — what if it's CPU bound?

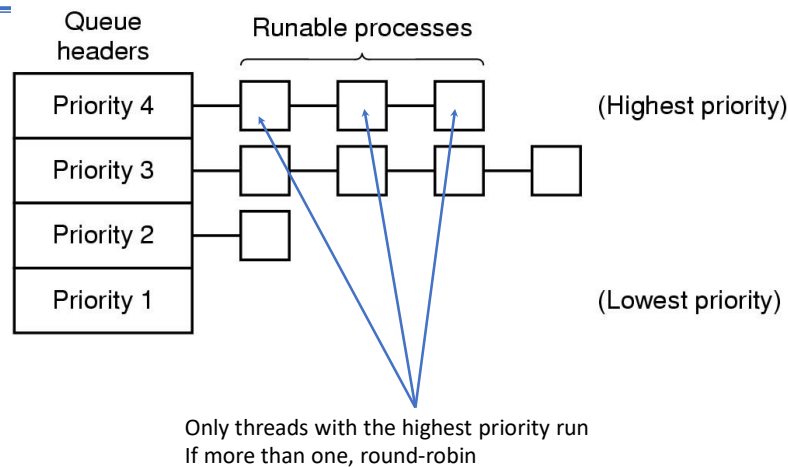
Spring 2026

Operating Systems

24

24

Priority Scheduling



Spring 2026

Operating Systems

25

25

Priority Inversion Revisited

- Strict priority scheduling can lead to a phenomenon called “priority inversion”
- Consider the following example where $p(H) > p(M) > p(L)$
 - H needs a lock currently held by L, so H blocks
 - M that was already on the ready list gets the processor before L
 - H indirectly waits for M
- On Path Finder, a watchdog timer noticed that H failed to run for some time, and continuously reset the system

Spring 2026

Operating Systems

26

26

Pintos: Priority Donation

- When a high priority thread H waits on a lock held by a lower priority thread L, donate H's priority to L and recall the donation once L releases the lock
- **Implement priority donation for locks**, not semaphores or conditional variables
- Handle the cases of multiple donations and nested donations

Spring 2026

Operating Systems

27

27

Priority Adjustments

- Periodically reduce the priority of the running process
- Eventually, it falls below the priority of the next process
- Alternative: increase the priority of non-running processes
- Called process aging
- Or do both

Spring 2026

Operating Systems

28

28

Dynamic Priority Adjustment

- Adjust process priority according to its recent history
 - Example: increase priority of non-running processes; decrease priority of running processes
- Boost priority of I/O-bound processes
- If process used $1/f$ of its last quantum, boost its priority proportional to f
- Use priority classes: have separate queues for each priority level, and run each queue round-robin; switch to lower-priority queue when this one is empty

Spring 2026

Operating Systems

29

29

Run Queues

- Each run queue is a linked list
- To raise or lower a process' priority, move it to a different list
- Two schemes for priority aging:
 - Not many processes: have a fine-grained counter for each process incremented at a clock interrupt; at some limit, increase priority
 - Lots of processes and queues: periodically, move each list to the next level up

Spring 2026

Operating Systems

30

30

Varying Quanta

- Many processes need just a little bit of CPU time
- Since process switches can be expensive, don't do them too often
- Solution to both problems: give lower priority queues longer quanta
- Top priority queue: one quantum
- Second queue: two quanta CPU allocation
- Third queue: four quanta, etc.
- Alternate solution: "short" (initial) quantum at high priority and "long" quanta at low priority thereafter

Spring 2026

Operating Systems

31

31

Helping Interactive Processes

- Look for signs of user input
- When they occur, give the process a very high priority
- Example: on CTSS, when a user typed a carriage return, the process got top priority
- Harder to do today — what's "interactive" on a networked process? For a mouse movement, which process is credited?

Spring 2026

Operating Systems

32

32

Gaming the System

- On time-sharing systems, watch for attempts to fake out the scheduler
- CTSS example: typing spurious carriage returns
- XDS 940 example: do really quick I/O operation
 - Solution: save remaining CPU quantum; when process restarts, use remainder instead of full allocation
- Not applicable on single-user machines — you're only hurting yourself
- Instead, watch for inadvertent influences on the wrong process

Spring 2026

Operating Systems

33

33

Process Priorities

- What processes should have higher priorities?
- Administrative issues
- System performance
 - Kernel processes (up to a point)
 - Interactive services processes (i.e., X server)
- Users can lower priority of their own processes, sometimes to avoid competing with themselves

Spring 2026

Operating Systems

34

34

Unix Priorities

- Tradition: lower numbers indicate higher priorities
- “Nice” value is a user-specified modifier
- A nice value of +20 specifies a very low priority process
- Only root can set negative niceness
- Default is 0
- Note: this is an API; internal metric can be different

Spring 2026

Operating Systems

35

35

Some Linux Priorities

UID	PRI	NI	SZ	CMD
root	76	0	606	init
root	94	19	0	[ksoftirqd/0]
root	75	0	0	[khubd]
root	83	0	0	[scsi_eh_1]
root	79	0	6285	ypbind
root	75	0	1242	/usr/sbin/sshd
smb	76	0	1007	ps

The PRI value factors in the niceness and the process' dynamic priority

Spring 2026

Operating Systems

36

36

Shortest Process Next

- Can we emulate batch systems' shortest first algorithm?
- It's hard, because we don't have good estimates
- Instead, use historical data for a moving, decaying, average
- Let first time = T_0 ; second is T_1

$$T_2 = \alpha T_0 + (1 - \alpha)T_1$$

$$T_3 = \alpha^2 T_0 + \alpha(1 - \alpha)T_1 + (1 - \alpha)T_2$$

$$T_4 = \alpha^3 T_0 + \alpha^2(1 - \alpha)T_1 + \alpha(1 - \alpha)T_2 + (1 - \alpha)T_3$$

...

Spring 2026

Operating Systems

37

37

Whose History?

- Do we measure command history?
- Hard; requires a lot of kernel state
- Besides, command time can vary a lot depending on input data
- Better to measure user (or terminal) behavior

Spring 2026

Operating Systems

38

38

Guaranteed Scheduling

- For n users or process on a system, give each $1/n$ of the CPU
- Measure actual CPU usage
- Calculate process' CPU time entitlement
- Look at the ratio of the two: 0.5 means it's had only half the CPU it's entitled to, so it gets priority over a process with a ratio of 2.0

Spring 2026

Operating Systems

39

39

Lottery Scheduling

- Give each process lottery tickets
- Higher priority processes get more tickets; lower priority process get fewer
- At scheduling time, pick a random ticket
- The process holding that ticket gets to run
- Note: tickets can be exchanged between processes, such as between client and server

Spring 2026

Operating Systems

40

40

Fair-Share Scheduling

- In some systems, processes don't compete, users do
- We don't want to encourage forking just to get a larger share of the CPU
- Solution: make decisions based on user CPU consumption instead of process CPU consumption
- Priorities, etc., can still apply

Spring 2026

Operating Systems

41

41

Real-Time Systems

- Must respond to actual clock-on-the-wall time
- A late process may be a useless process
- Two types, hard and soft
- Hard real time systems have deadlines that must be met; used for process control, avionics, etc.
- Soft real time tries its best, but can miss occasional deadlines
- Both depend on knowledge of processing time per request

Spring 2026

Operating Systems

42

42

Periodic Events

- Many real-time events occur with a regular frequency
- Suppose there are m events, with the event i needing C_i seconds of CPU and occurring every P_i seconds
- System works if and only if

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

- Watch out for process switch overhead. . .

Spring 2026

Operating Systems

43

43

Aperiodic Events

- Other events don't happen on a regular schedule
- The basic constraint is the same: total load must be less than total capacity
- Engineering such systems is harder

Spring 2026

Operating Systems

44

44

Mixing Real-Time and Non-Realtime Processes

- Some systems have both kinds of processes (and schedulers)
- General strategy: give real-time processes priority over all conventional processes
- Schedule them round-robin or possibly even non-preemptively

Spring 2026

Operating Systems

45

45

Multiple Real-Time Processes

- A runs every 30 msec; each time it needs 10 msec of CPU time
- B runs 25 times/sec for 15 msec
- C runs 20 times/sec for 5 msec
- For our equation, A uses $10/30$ of the CPU, B uses $15/40$, and C uses $5/50$; that's about 81%

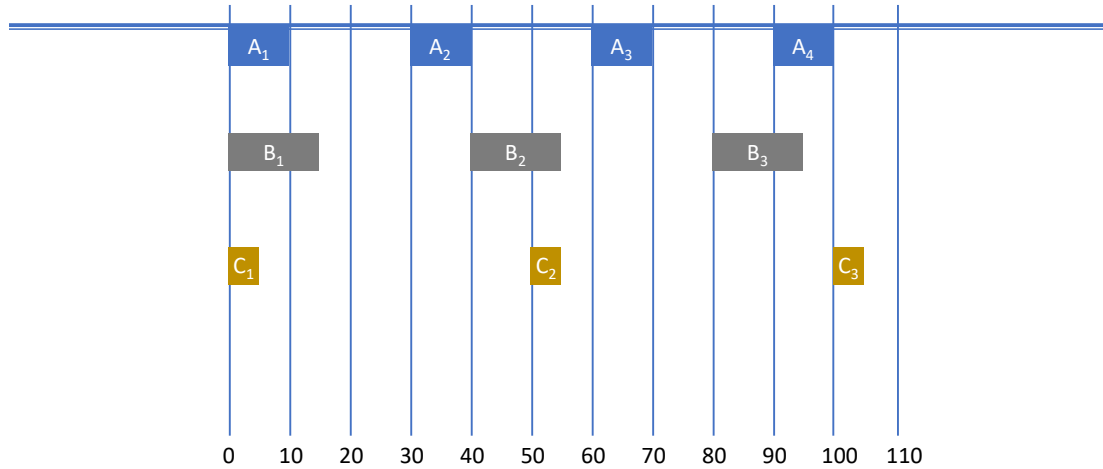
Spring 2026

Operating Systems

46

46

Diagram



A_1 must finish before A_2 starts, B_1 before B_2 , . . .

Spring 2026

Operating Systems

47

47

Other Issues

- Some real-time systems permit preemption; some do not
- Desirability depends on system type (text's discussion is for multimedia system, which are usually pre-emptible)
- May have aperiodic processes in the mix
- Static or dynamic scheduling

Spring 2026

Operating Systems

48

48

Rate Monotonic Scheduling

- A static scheduling algorithm by Lieu and Layland (1973)
- Conditions:
 1. Each periodic process completes within its slot
 2. No interprocess dependencies
 3. Each process needs the same amount of CPU each time
 4. Non-periodic processes have no deadlines
 5. Preemption happens instantly with no overhead
- Yes, this is an oversimplified model. . .

Spring 2026

Operating Systems

49

49

Algorithm

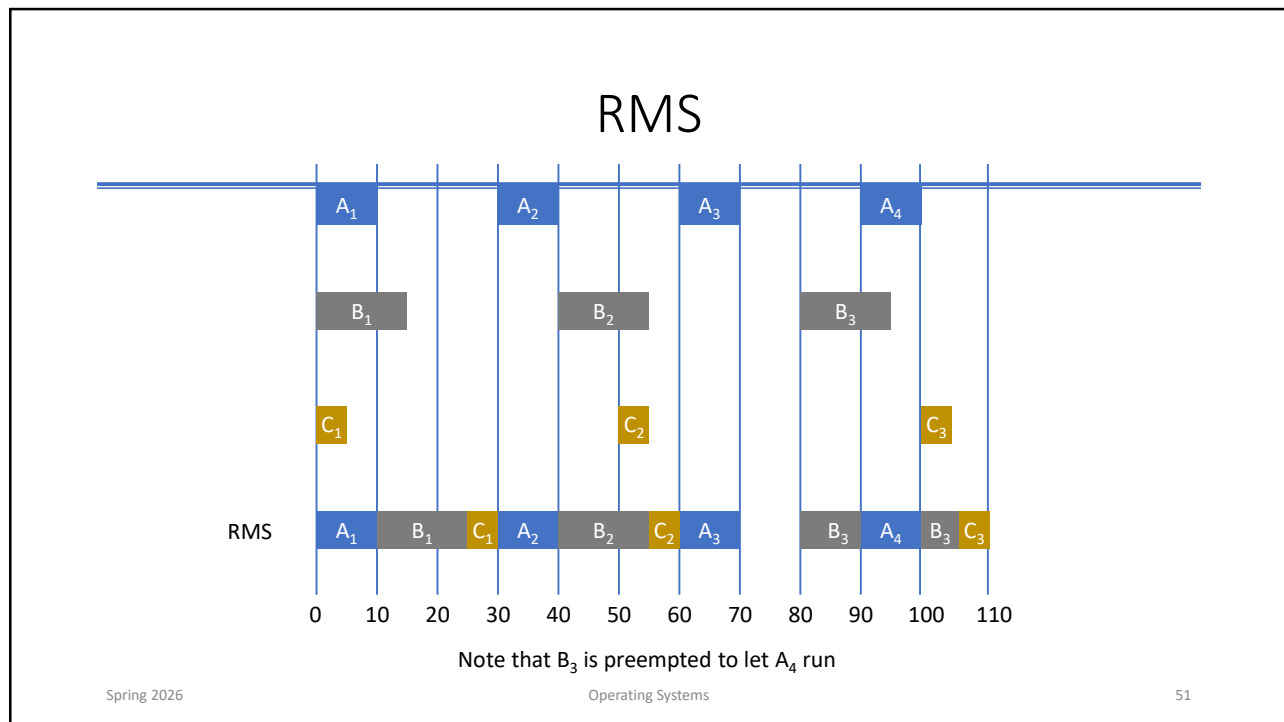
- Assign a process priority equal to its frequency:
 $A = 33, B = 25, C = 20$
- Always run the highest-priority runnable process
- Thus, A can preempt B or C; B can preempt C
- Proved optimal among class of static algorithms

Spring 2026

Operating Systems

50

50



51

Earliest Deadline First

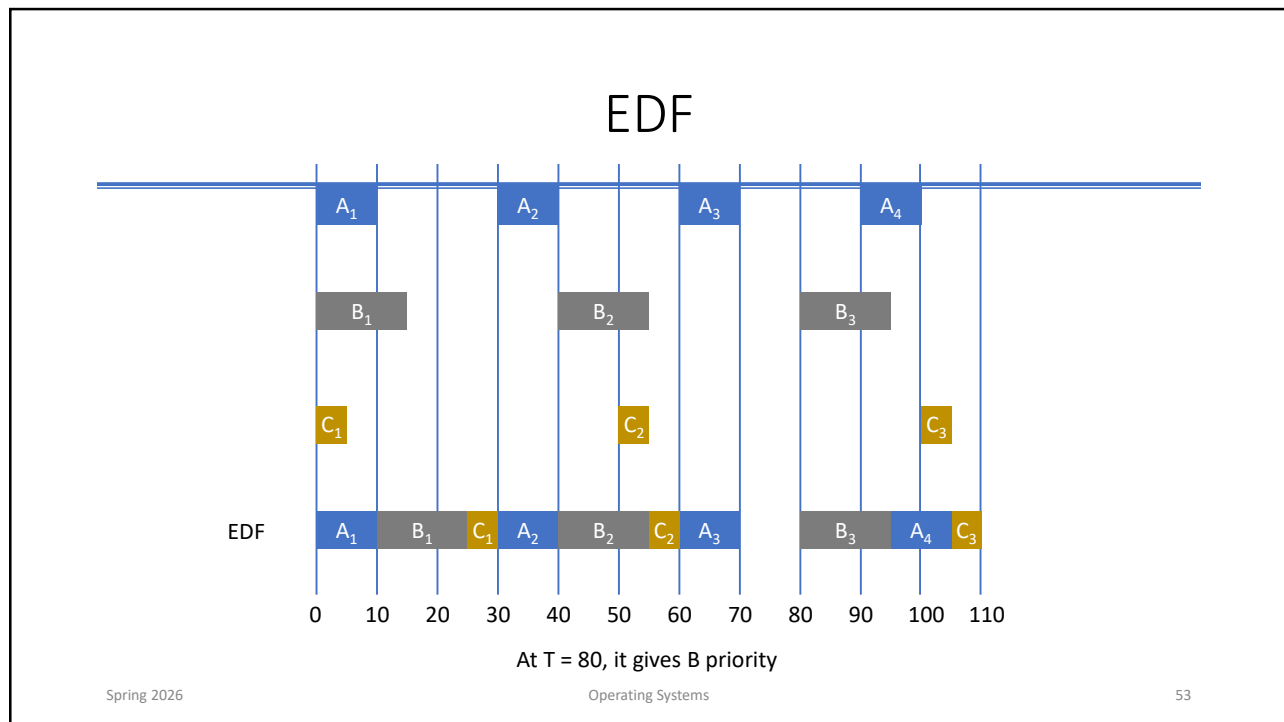
- More general model
- Supports aperiodic events, non-identical CPU bursts
- Dynamic priority assignment
- When a process starts, it announces its deadline
- Priorities are assigned in order of deadline
- Initially, A goes first, because it has to finish by $T = 30$; B's deadline is $T = 40$ and C's is $T = 50$

Spring 2026

Operating Systems

52

52



53

RMS Doesn't Always Work

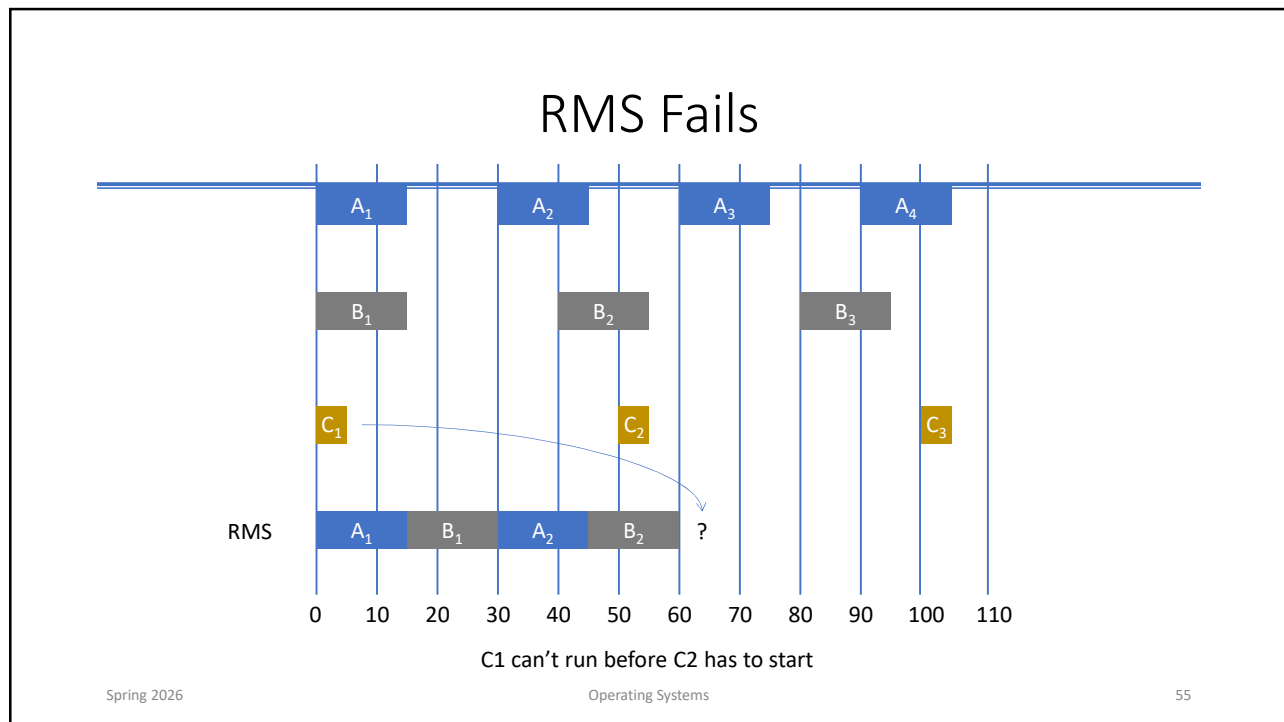
- Suppose that A needs 15 msec each time
- Our formula says we're ok: $(15/30 + 15/40 + 5/50) = 97.5\%$
- But it fails
- A1 runs from T = 0 to T = 15; B1 runs from T = 15 to T = 30
- At that point, A2 is ready, and has a higher priority than C1; B2 follows it
- There's no time for C1 before C2 has to start

Spring 2026

Operating Systems

54

54



55

Why Did it Fail?

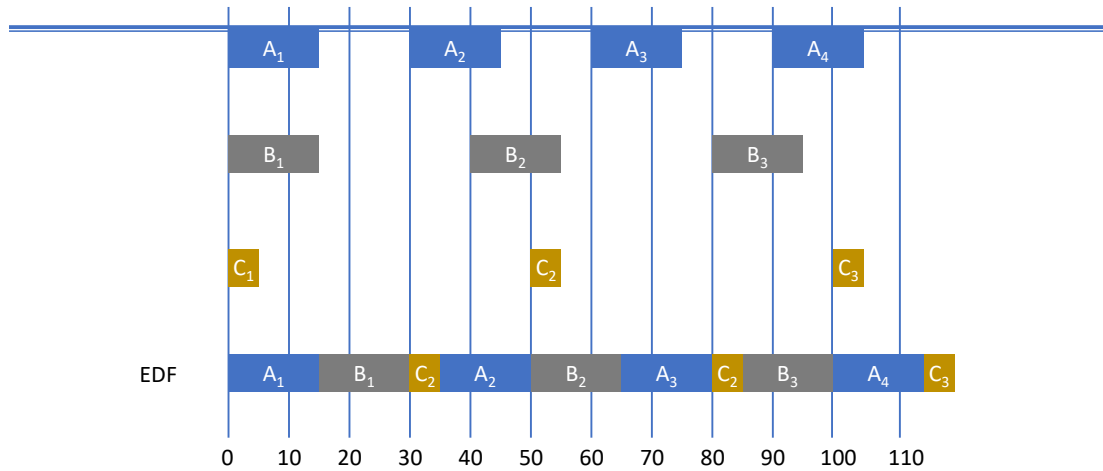
- RMS is only guaranteed to work if

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq m(2^{1/m} - 1)$$
- As $m \rightarrow \infty$, utilization approaches $\ln 2 = 0.693$
- For $m = 3$, it can fail (though won't always) at 78%
- Maximum allowed utilization goes down as m increases
- EDF will succeed for this example
- The CPU idle period — 2.5% — will occur every 200 msec

Spring 2026 Operating Systems 56

56

EDF Succeeds



Spring 2026

Operating Systems

57

57

Gantt Charts

- Gantt Charts – useful tool for modeling process scheduling
- Especially useful with an automated tool
- Other forms...

	1	2	3	4	5	6	7	8
P_1	* ?							
P_2								
P_3								
P_4								
P_5								

Spring 2026

Operating Systems

58

58

Other Issues – Scheduling Threads

- With user-level threads, there is no interaction with the scheduler
- A kernel-level thread implementation relies on the system's scheduler
- It's often beneficial to schedule several threads from the same process consecutively — avoid changes to the memory map
- Similar logic says keep threads from the same process on a single CPU, if feasible
- Application-level thread scheduler can handle priorities more easily, though kernel-level priorities aren't hard to set

Spring 2026

Operating Systems

59

59

Other Issues – Scheduling and Multiprocessors

- What processes run on which processor?
- Does it matter?
- What are the implications?

Spring 2026

Operating Systems

60

60

Asymmetric Multiprocessing

- All kernel functions are handled by the master CPU
- The only thing the other CPUs do in the kernel is pull processes off the run queue and do context switches
- Simplifies OS design — locking is much simpler
- Common first step in OS conversion for multiprocessor use

Spring 2026

Operating Systems

61

61

Symmetric Multiprocessing

- Each processor can do anything
- Possible to have more than one CPU in the kernel simultaneously
- Need fine-grained locking; single “big lock” is almost the same as asymmetric
- Message Passing
- Locking is a very important issue for multiprocessors

Spring 2026

Operating Systems

62

62

Processor Affinity

- Sometimes, it's better (or necessary) for a given process to execute on a specific CPU
- Example: memory map (and cache)
- I/O issues — sometimes a specific I/O device is on a local bus

Spring 2026

Operating Systems

63

63

How Do We Measure CPU Time?

- Early clocks were low resolution
- Unix classic: 60 Hz
- Too coarse to measure a fraction of a quantum
- Besides, the clock was an I/O device, hence slow to access

Spring 2026

Operating Systems

64

64

Statistical Time

- At each clock tick, add 1 to the current process' CPU counter
- Actually, two counters, one for user mode and one for kernel mode
- Not accurate over short periods:
 - A process may run for too short a time and not get charged
 - A process may start its quantum right before the timer tick and be charged too much
- Statistically, though, it's good enough

Spring 2026

Operating Systems

65

65

I/O and Memory

- CPU time isn't the only scarce resource
- Especially today, total system performance is limited by I/O bandwidth and memory availability
- Must read programs in from disk
- Must have memory for them
- That may mean paging out another process, which puts more load on the disk
- The scheduler should interact with the I/O subsystem and the memory subsystem

Spring 2026

Operating Systems

66

66

Different Schedulers – Scheduler Algorithms

- Modern systems may have many processes running
- A cluster might be running 525 processes
- Even on modern CPUs, we don't want scheduling algorithms that iterate over all processes
- Linux uses a $O(1)$ scheduler — scheduling decisions take constant time, regardless of the number of processes

Spring 2026

Operating Systems

67

67

Policy versus Mechanism

- Put some basic mechanism (or mechanisms) in the kernel
- Permit user processes to set parameters that control scheduling
- Simplest example: nice command
 - `nice -n 19 tar cvzf archive.tgz largefile`
- Solaris permits much more control: three classes of scheduler, and parameters within that class

Spring 2026

Operating Systems

68

68

Solaris Scheduler

- Scheduler classes: real-time, time-sharing, interactive
- Parameters:
 - Real-Time priority, quantum
 - Kernel threads (System only)
 - Timesharing priority, priority limit
 - Interactive priority, priority limit
- Newer versions of Solaris have fair-share scheduling and fixed-priority scheduling

Spring 2026

Operating Systems

69

69

Solaris Priorities

Priority	Quantum	New Priority	After Sleep
0	200	0	50
5	200	0	50
15	160	5	51
50	40	40	58
55	40	45	58
59	20	49	59

Higher priority processes get shorter quanta. Process get a lower priority when they use up their quantum, and higher priority after blocking for I/O.

Spring 2026

Operating Systems

70

70

Linux Scheduler

- Both real-time and priority
- Real-time scheduling can be round-robin or FCFS
- Dynamic timeslice (quantum) computation
- Kernel preemption possible if no locks are held

Spring 2026

Operating Systems

71

71

Windows XP

- Real-time and priority scheduling
- Priority classes: Real-time, High, Above Normal, Normal, Below Normal, Idle
- Relative priority within class: Time-critical, Highest, Above Normal, Normal, Below Normal, Idle
- Effective priority calculated from this matrix
- Priority lowered on quantum expiration
- Extra priority boost for process associated with current window
- Unix can't easily do that — the window manager knows nothing of processes

Spring 2026

Operating Systems

72

72

Evaluating Scheduler Algorithms

- First question: what criteria do you want to optimize for?
- Possibilities include CPU utilization, responsiveness, real-time scheduling, etc.
- Several ways to do the evaluation

Spring 2026

Operating Systems

73

73

Deterministic Modeling

- Start with a specific workload, i.e., of process' CPU demands and arrival times
- Model them with Gantt charts, as we've seen
- Evaluate according to desired metric
- Simple and fast — and only useful for loads that look a lot like what you model

Spring 2026

Operating Systems

74

74

Queueing Theory

- Start with probability distributions of CPU requests, arrival times, etc.
- Common assumption: arrivals are distributed according to a Poisson distribution

- Sample (and simple) result:

Let n be the average queue length, W the average queue wait time, and λ the mean interarrival time (regardless of distribution). Then

$$n = \lambda W$$

Spring 2026

Operating Systems

75

75

Simulation

- Build a simulator of the scheduling algorithm
- Feed in simulated inputs and see what happens
- Simulations can be fed by probability distributions or by trace data from real systems
- Such trace data is an excellent way to compare two different simulators

Spring 2026

Operating Systems

76

76

Metacomment

- Trace data is always useful
- Instruction traces, network packet traces, CPU load traces, etc.
- Some such datasets become the way to evaluate new schemes

Spring 2026

Operating Systems

77

77

Build It and Try It

- Build a real system and see what happens
- But what's the load? Real users?
- If you're lucky, you have trace data to feed in (and a system amenable to such replays)

Spring 2026

Operating Systems

78

78

Limits of Evaluation

- Load changes
 - Sometimes, load changes because of scheduler changes
 - Example: if a process gets the CPU more quickly after a disk I/O request, it may be able to issue the next request within the rotational delay of the disk
- Conclusion: we need flexible scheduling algorithms that can be tuned at each site

Spring 2026

Operating Systems

79

79

Summary

- Scheduling is a complex matter
- The criteria and algorithms have changed somewhat, but the problem remains
- Any time you click on something and it doesn't respond immediately, there's a scheduler problem

Spring 2026

Operating Systems

80

80

See you next time!